Backend Engineering Assignments (Bun + TypeScript)

Prepared by: Senior Backend Engineer (10+ Years Industry Experience)
Objective: These assignments are designed to build strong fundamentals in backend engineering, API design, data validation, error handling, and testing using Bun and TypeScript.

Assignment 1: Basic REST API with Health Check

Objective: Learn how to set up a basic HTTP server using Bun and TypeScript.

Requirements:

- Use Bun's native HTTP server or a minimal framework.

- Create a GET endpoint: /health

- Response (200 OK):

{ "status": "ok", "timestamp": "" }

Constraints: No database required.

Test Cases:

- GET /health returns status code 200.

- Response body contains 'status' with value 'ok'.

- 'timestamp' is a valid ISO-8601 string.

Hint: Look up Bun.serve() documentation.

Assignment 2: User Creation API with Validation

Objective: Learn request body parsing and validation.

Endpoint: POST /users

Request Body (JSON):

{ "name": "string", "email": "string", "age": number }

Validation Rules:

- name: required, min 3 characters
- email: required, valid email format
- age: optional, must be >= 18 if provided

Success Response (201):

{ "id": "uuid", "name": "...", "email": "...", "age": number | null }

Error Response (400):

{ "error": "Validation error message" }

Test Cases:

- Valid payload returns 201 and generated id.
- Missing name returns 400.
- Invalid email returns 400.

Hint: Search for schema validation libraries compatible with Bun.

Assignment 3: In-Memory Data Storage and Retrieval

Objective: Understand state management in backend services.

Endpoints:

- POST /products
- GET /products/:id

Product Body:

{ "name": "string", "price": number }

Rules:

- price must be > 0
- Store products in memory (Map or object)

Responses:

- POST returns 201 with stored product including id
- GET returns 200 with product or 404 if not found

Test Cases:

- Create product and retrieve it successfully.
- Request non-existing product returns 404.

Hint: Think about how server restarts affect in-memory data.

Assignment 4: Error Handling and Middleware Pattern

Objective: Build consistent error handling.

Task:

- Implement a centralized error handler.

- Ensure all errors follow the same response format.

Error Format:

{ "error": { "message": "string", "code": "string" } }

Scenarios:

- Validation error

- Not found error

- Internal server error

Test Cases:

- Trigger each error type and verify format.

- Ensure no stack traces are leaked.

Hint: Research middleware patterns even if not using a framework.

Assignment 5: Automated Testing with Bun Test Runner

Objective: Learn how to write automated backend tests.

Task:

- Write tests for Assignment 2 and 3 APIs.

- Use Bun's built-in test runner.

Requirements:

- At least 5 test cases.

- Cover both success and failure scenarios.

Test Areas:

- Status codes

- Response body structure

- Validation errors

Grading Criteria:

- Tests should be deterministic.

- Clear naming and structure.

Hint: Search 'bun test http server' for examples.